

MEMORIA ESCRITA DE TYPE-PEPTIDE

(Documentación técnica)

1. Resumen ejecutivo

Type-Peptide es una aplicación web para visualizar péptidos como hélices α ideales e inspeccionar rápidamente sus propiedades fisicoquímicas.

La herramienta se compone de:

- Un **frontend estático** (HTML/CSS/JS) servido por Nginx.
- Una **API mínima Node/Express** cuya función principal es exponer endpoints de health y metadatos del despliegue.

El usuario ingresa una secuencia de aminoácidos y, completamente en el lado del cliente, se calculan propiedades como:

- Carga neta a pH neutro.
- Punto isoeléctrico.
- Porcentaje de residuos hidrofóbicos.
- Índice de Boman.
- Momento hidrofóbico relativo.
- Índice de Wimley White.

Al mismo tiempo, se construye una hélice 3D con Three.js donde cada residuo se representa por una esfera y un sprite de cadena lateral.

2. Objetivos

- Proporcionar una visualización interactiva y pedagógica de péptidos.
- Integrar cálculos fisicoquímicos clásicos de forma accesible para estudiantes/investigadores.
- Mantener una arquitectura sencilla, fácil de desplegar en entornos de laboratorio o académicos (Nginx + Node en Docker).
- Facilitar la extensión futura (ej. conexión con bases de datos de péptidos).

3. Alcance

La versión actual de Type-Peptide permite:

- Ingresar una secuencia de hasta 200 aminoácidos en código de una letra.
- Validar que solo contenga aminoácidos estándar (20 residuos).
- Visualizar la secuencia como una hélice α con:
 - Colores y radios dependientes del tipo de residuo.
 - Etiquetas con la letra.

- Sprites 2D de cadenas laterales y nombres completos al hacer clic.
- Consultar propiedades fisicoquímicas a nivel global del péptido.
- Ajustar la vista mediante controles de cámara (bloqueo, desbloqueo, centrado).

No incluye:

- Persistencia de secuencias en servidor.
- Exportación de resultados.
- Integración con bases de datos externas (aunque la API está preparada para futuras extensiones).

4. Arquitectura

4.1. Frontend (Nginx)

- Nginx sirve contenido estático desde `/usr/share/nginx/html`:
 - `index.html`
 - `styles.css`
 - `app.js`
 - `peptide-utils.js`
 - Librerías: `three.min.js`, `TrackballControls.js`, `Detector.js`, `colorHandler.js`
 - Imágenes en `assets/img/`.
- Configuración relevante en `nginx.conf`:
 - `server_name type-peptide.medellin.unal.edu.co`;
 - `location = /health` → retorna `ok\n`.
 - `location /api/` → `proxy_pass http://api:8052`;
 - Políticas de caché:
 - HTML/JS/CSS sin caché agresiva (para facilitar despliegues).
 - Imágenes y fuentes con expiración prolongada.

4.2. API (Node.js/Express) — `api/server.js`

- Usa `dotenv` para cargar variables desde `.env` (en desarrollo).
- Endpoints:
 - `GET /health`
 - Comprueba existencia de `FASTA_PATH` (por defecto `/app/sequences/BD_FINAL.fasta`).
 - Devuelve `{ status, fastaPath, fastaExists }`.
 - `GET /api/info`
 - Devuelve:
 - Nombre y versión de la API.
 - Valores de entorno relevantes: `PORT`, `FASTA_PATH`, `FORCE_FASTA_ONLY`.
 - Middleware final `404` → `JSON { error: 'Not found' }`.

- Puerto: **PORT** (por defecto 8051 o 8052 según configuración).

Esta API funciona principalmente como punto de health y metadatos, dejando toda la lógica de cálculo y visualización en el navegador.

4.3. Contenedores Docker

- **docker-compose.yml:**
 - Servicio **api**:
 - **build:** ./api.
 - **env_file:** ./api/.env.
 - **expose:** "8052".
 - Volumen solo lectura: ./api/sequences:/app/sequences:ro.
 - **healthcheck** con **wget** -q0-http://localhost:8052/health.
 - Servicio **frontend**:
 - Imagen base **nginx:alpine**.
 - Volúmenes:
 - ./frontend:/usr/share/nginx/html:ro
 - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
 - **expose:** "80".
 - Red externa: **inverpep_default**.

5. Estructura del repositorio

Ejemplo (adaptado a Type-Peptide):

```
type-peptide/
├── api/
│   ├── sequences/
│   │   └── BD_FINAL.fasta
│   ├── .env          # (solo desarrollo)
│   ├── Dockerfile
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── assets/
│   │   ├── img/
│   │   │   ├── A.png
│   │   │   ├── C.png
│   │   │   └── ...
│   │   └── logo-typepeptide.png
│   └── assets/
│       └── js/
```

```

|   |   |   |
|   |   |   |— three.min.js
|   |   |   |— TrackballControls.js
|   |   |   |— Detector.js
|   |   |   |— colorHandler.js
|   |   |— app.js
|   |   |— peptide-utils.js
|   |   |— index.html
|   |   |— styles.css
|— docker-compose.yml
|— nginx.conf
|— README.md

```

Descripción breve:

- **api/server.js**: API Node/Express (health, info).
- **frontend/index.html**: estructura de la interfaz, modal de ayuda, paneles y canvas.
- **frontend/app.js**: lógica de Three.js y de la UI.
- **frontend/peptide-utils.js**: funciones de cálculo de propiedades.
- **frontend/styles.css**: estilos de layout, tabla y modal.
- **assets/js/**: librerías de terceros y lógica de color.
- **assets/img/**: sprites de cadenas laterales y logo.

6. Detalle de módulos

6.1. frontend/app.js

Responsable de:

- Inicializar escena Three.js:
 - `scene`, `camera`, `renderer`, `controls`.
 - Luces: ambiente + dos focos (`SpotLight`) en direcciones opuestas.
- Manejo de listas:
 - `sphereList`: referencia a las esferas de cada aminoácido, usadas para raycasting.
 - `groupList`: lista de grupos 3D (esfera + textos + sprite) por posición.
- Carga de texturas de cadenas laterales:
 - Función `loadTextures("assets/img/")`:
 - Carga `A.png`, `C.png`, ..., `Y.png`.
 - Crea `SpriteMaterial` para cada uno.
- Eventos:
 - `click` sobre el canvas:
 - Calcula intersección con `sphereList` vía `Raycaster`.
 - Alterna visibilidad de sprite lateral y texto nombre/posición.
 - `resize` de ventana:
 - Recalcula tamaño del renderer y aspect ratio de la cámara.
 - Botones:

- `btnLock / btnUnlock` → `setCamMode('lock' | 'unlock')`.
 - `tp-center-btn` → `getCamara1()` (centrar).
 - `tp-search-btn` → dispara `keyup` en el input.
- Redibujo de la hélice:
 - Función `draw()`:
 - Borra la línea previa, si existe.
 - Recorre la secuencia y:
 - Crea o reutiliza grupos para cada posición.
 - Ajusta geometría de la hélice mediante `get_x` y `get_y`.
 - Elimina grupos sobrantes si la nueva secuencia es más corta.
- Sprites de texto:
 - `makeTextSprite(message, parameters)`: crea etiquetas a partir de un `canvas` 2D y las convierte en textura de sprite.
- Exposición de funciones globales de cámara:
 - `unlookCamara()`, `lookCamara()`, `getCamara1()`.

6.2. frontend/peptide-utils.js

Define:

- Lista de aminoácidos válidos.
- Nombres completos por letra (`aminosFullName`).
- Conversión de ángulos a radianes (`toRadians`).
- Funciones geométricas de la hélice:
 - `get_x(i, pos_off = 0, neg_off = 0)`
 - `get_y(i, pos_off = 0, neg_off = 0)`
 - Ambas usan un radio base de 10 y un desplazamiento angular de 100° por residuo.
- Validación de entrada (`validateEntry`): permite solo letras de aminoácidos + teclas de borrado.
- Funciones fisicoquímicas:
 - `getCharge(seq, pH=7.0)`: calcula la carga neta considerando pKas estándar y extremos.
 - `getIsoEle(seq)`: barrido de pH 0–14 paso 0.01.
 - `getHidrof(seq)`: % de residuos hidrofóbicos.
 - `getMoment(seq)`: momento hidrofóbico relativo.
 - `getBoman(seq)`: índice de Boman.
 - `getWimley(seq, scale)`: suma de valores en una escala Wimley (octanol/interfaz por defecto).
- Escalas Wimley:
 - `octanol_interface_wimley`, `octanol_wimley`, `interface_wimley`.

Estos cálculos se realizan completamente en el navegador y se utilizan para actualizar la tabla del panel izquierdo.

7. Flujo de datos en el cliente

1. El usuario accede a la URL de Type-Peptide.
2. Nginx sirve `index.html`, `styles.css`, `three.min.js`, `TrackballControls.js`, `app.js`, `peptide-utils.js` y las imágenes necesarias.
3. Al cargarse el DOM:
 - `app.js` inicializa la escena y la tabla (todo en cero).
4. El usuario escribe una secuencia:
 - Evento `keyup` en `#aminoacid`:
 - Convierte a mayúsculas.
 - Recalcula propiedades usando `peptide-utils.js`.
 - Actualiza la tabla.
 - Llama a `draw()` para reconstruir la hélice.
5. El usuario interactúa con la escena:
 - Movimiento de cámara mediante `TrackballControls`.
 - Click en residuos para mostrar sprites y nombres.

La API solo se consulta para healthchecks (si el frontend o un monitor lo requieren); no interviene en el flujo de cálculo.

8. Configuración (variables de entorno)

API (`api/.env` o `docker-compose.yml`):

- `PORT=8052`
- `FASTA_PATH=/app/sequences/BD_FINAL.fasta`
- `FORCE_FASTA_ONLY=false`
- `DB_CONNECTION`, `DB_HOST`, `DB_PORT`, `DB_DATABASE`, `DB_USERNAME`, `DB_PASSWORD`, `TABLE_NAME`, `HEADER_COLUMN`, `SEQUENCE_COLUMN`
(reservadas para futuras extensiones con base de datos).

Nginx / frontend:

- `VIRTUAL_HOST`
- `LETSENCRYPT_HOST`
- `LETSENCRYPT_EMAIL`
- `VIRTUAL_PORT=80`

9. Despliegue

En la raíz del proyecto:

```
docker compose build --no-cache api
```

```
docker compose up -d
```

Ver logs:

`docker compose logs -f api`

`docker compose logs -f frontend`

10. Rendimiento y decisiones de diseño

- **Cálculos en cliente:**
 - Para secuencias de hasta 200 residuos, el costo computacional es bajo.
 - Evita carga en el servidor y permite escalado sencillo.
- **Renderizado continuo:**
 - `requestAnimationFrame(animate)` mantiene actualizada la escena.
 - `TrackballControls` se actualiza en cada frame.
- **Sprites vs. modelos 3D completos:**
 - Se optó por sprites 2D para las cadenas laterales:
 - Menor complejidad y carga de GPU.
 - Suficiente para un uso pedagógico.

11. Seguridad

- La aplicación no almacena ni envía las secuencias a un backend (salvo healthchecks de prueba).
- No hay autenticación ni manejo de datos sensibles.
- Nginx puede configurarse para servir sobre HTTPS usando Let's Encrypt y un reverse proxy externo.

12. Respaldo y restauración

- Código fuente: repositorio del proyecto.
- Configuración:
 - `docker-compose.yml`
 - `nginx.conf`
 - `api/.env` (no versionarlo en repositorios públicos).
- Recursos estáticos:
 - `frontend/assets/img/*`

Restauración: clonar proyecto, restaurar archivos de configuración y reconstruir contenedores.

13. Mantenimiento y evolución

- Actualizaciones menores:
 - Ajustes en `styles.css`, `index.html` o `app.js` para mejorar la UX.
- Actualizaciones mayores:
 - Añadir nuevas propiedades fisicoquímicas en `peptide-utils.js`.

- Integrar con bases de datos mediante nuevos endpoints en `api/server.js`.
- Monitoreo:
 - Uso de `GET /health` (Nginx) y `GET /health` (API) para supervisión automática.

14. Limitaciones conocidas

- No maneja péptidos con aminoácidos no estándar o modificaciones postraduccionales.
- La hélice es idealizada (100° por residuo, radio fijo), no se trata de una estructura derivada de simulaciones o PDB.
- No existe exportación directa de la escena 3D (por ejemplo, como imagen o archivo 3D).

15. Licencia y autores

Licencia

Este proyecto se distribuye bajo una licencia de software definida por la institución responsable. En ausencia de un texto específico en el repositorio, se recomienda adoptar una licencia abierta (por ejemplo, MIT o GPL) que permita su uso académico y de investigación, manteniendo los créditos correspondientes.

Autores y créditos

- Autora principal: Hinara Pastora Sánchez Mata.
Información de contacto: 3044822338, hisanchezm@unal.edu.co
- Crédito institucional:
“Desarrollado por la línea de interacciones moleculares — Biología Funcional, Facultad de Ciencias, Universidad Nacional de Colombia, Sede Medellín”.